EECE5644 Fall 2026 – Assignment 3

David Abu

All experiments were implemented in Python 3 (NumPy, SciPy, scikit-learn, joblib, Matplotlib) with fixed random seeds, so every number below is reproducible from the code in the Appendix.

Question 1: MLP approximation of class posteriors

Data distribution

There are $C = 4$ classes with uniform priors $P(L = c) = 1/4$ and Gaussian class-conditional pdfs $p(\mathbf{x}|L = c) = \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_c, \Sigma_c)$, $\mathbf{x} \in \mathbb{R}^3$. The means sit on the vertices of a regular tetrahedron scaled by $s = 0.85$:

$$\boldsymbol{\mu}_1 = s[1, 1, 1]^T, \quad \boldsymbol{\mu}_2 = s[1, -1, -1]^T, \quad \boldsymbol{\mu}_3 = s[-1, 1, -1]^T, \quad \boldsymbol{\mu}_4 = s[-1, -1, 1]^T.$$

Each covariance is $\Sigma_c = R_c \,\mathrm{diag}(\boldsymbol{\lambda}_c)\, R_c^T$ with a different rotation R_c and eigenvalues $\boldsymbol{\lambda}_1 = (1.0, 0.6, 0.3)$, $\boldsymbol{\lambda}_2 = (0.8, 0.8, 0.4)$, $\boldsymbol{\lambda}_3 = (1.2, 0.5, 0.5)$, $\boldsymbol{\lambda}_4 = (0.7, 0.6, 0.9)$, so the classes have different shapes and orientations and the optimal decision boundaries are quadratic. The rotation angles are listed in `q1.py`. The scale s was tuned by Monte Carlo (200,000 samples per candidate): $s = 0.8, 0.9, 1.0$ gave Bayes errors of 17.2%, 12.8% and 9.1%, so $s = 0.85$ places the optimal error near the middle of the required 10–20% range.

Training sets with $N \in \{100, 500, 1000, 5000, 10000\}$ samples and a test set with 100,000 samples were drawn independently. The test set was used only for final performance evaluation. Its class counts are 24973, 25152, 25003, 24872.

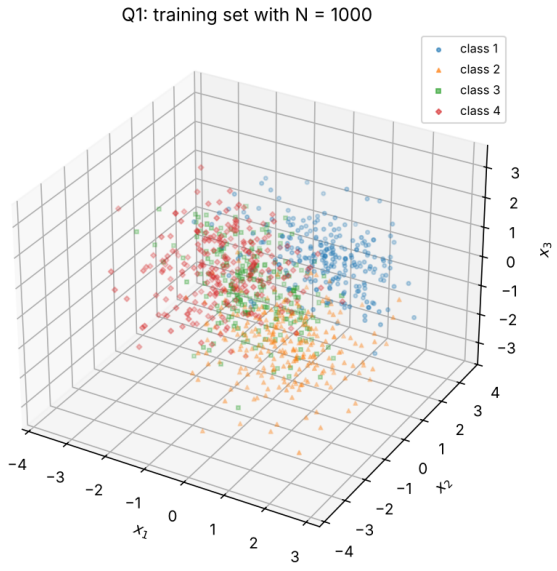


Figure 1: The $N = 1000$ training set.

Theoretically optimal classifier

With 0–1 loss, the minimum-expected-risk rule is the MAP rule

$$D(\mathbf{x}) = \arg\max_c \; P(L = c|\mathbf{x}) = \arg\max_c \; \Big[\ln \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_c, \Sigma_c) + \ln P(L = c)\Big],$$

which, with uniform priors, reduces to choosing the largest class-conditional log-likelihood. Applied to the test set with the true pdfs, it gives

$$\boxed{\hat{P}_{\text{opt}}(\text{error}) = 0.1484}$$

This is the aspirational performance level for the MLPs. Its confusion counts are:

	$L = 1$	$L = 2$	$L = 3$	$L = 4$
$D = 1$	21665	1681	615	1562
$D = 2$	1197	20998	1000	1488
$D = 3$	474	834	22229	1555
$D = 4$	1637	1639	1159	20267

Table 1: Optimal classifier on the test set: number of samples with decision $D = i$ (rows) and true label $L = j$ (columns).

MLP model

Each model is a two-layer MLP: P hidden perceptrons with the smooth-ReLU (softplus) activation $\phi(a) = \ln(1 + e^a)$, followed by a softmax output layer:

$$\mathbf{h} = \phi\big(W_1\tilde{\mathbf{x}} + \mathbf{b}_1\big) \in \mathbb{R}^P, \qquad \hat{P}(L = c|\mathbf{x}) = y_c(\mathbf{x}) = \frac{\exp\left(\mathbf{w}_{2,c}^T\mathbf{h} + b_{2,c}\right)}{\sum_{k=1}^{C}\exp\left(\mathbf{w}_{2,k}^T\mathbf{h} + b_{2,k}\right)}.$$

Here $\tilde{\mathbf{x}}$ is $\mathbf{x}$ standardized with the mean and standard deviation of the data the model is trained on, so validation and test data are never used for standardization. The model has $P(d{+}1){+}C(P{+}1) = 8P{+}4$ parameters $\boldsymbol{\theta}$. The softmax guarantees $y_c > 0$ and $\sum_c y_c = 1$.

Maximum-likelihood training. For iid training data $\{(\mathbf{x}_n, l_n)\}_{n=1}^N$ with one-hot labels t_{nc}, the likelihood of the labels under the model is $\prod_n \prod_c y_c(\mathbf{x}_n)^{t_{nc}}$. So the ML estimate minimizes the average cross-entropy

$$\hat{\boldsymbol{\theta}}_{ML} = \arg\min_{\boldsymbol{\theta}} \; -\frac{1}{N} \sum_{n=1}^{N} \sum_{c=1}^{C} t_{nc} \ln y_c(\mathbf{x}_n; \boldsymbol{\theta}).$$

No regularization term is added, so this is pure ML as the question specifies. The objective is minimized with full-batch L-BFGS (`scipy.optimize.minimize`, method `L-BFGS-B`). The gradient is computed analytically by backpropagation: $\partial/\partial \mathbf{a}_2 = (\mathbf{y} - \mathbf{t})/N$ at the softmax, then back through W_2 and the softplus derivative $\phi'(a) = \sigma(a) = 1/(1 + e^{-a})$. Weights are initialized randomly (Glorot-style normal draws), with a fresh draw for every restart.

Local optima. The cross-entropy of an MLP is non-convex. Every training call therefore runs several independent random initializations and keeps the solution with the *highest training log-likelihood* (lowest average cross-entropy). The final models use 10 restarts (up to 3000 L-BFGS iterations each). The cross-validation fits use 2 restarts (up to 1000 iterations each) to keep the computation manageable.

Classification. Treating $y_c(\mathbf{x})$ as the class posterior, the MAP rule is $D(\mathbf{x}) = \arg\max_c y_c(\mathbf{x})$.

Model-order selection by 10-fold cross-validation

For each training set, the candidate hidden-layer sizes were $P \in \{1, 2, 3, 4, 6, 8, 12, 16, 24, 32\}$. The training set was shuffled and split into 10 disjoint folds (`sklearn.model_selection.KFold`). For each P and each fold, an MLP was trained by ML on the other 9 folds and used to classify the held-out fold. The CV objective is the total number of validation errors summed over the 10 folds, divided by N: an estimate of P(error) in which every sample is classified exactly once by a model that did not see it. The selected P minimizes this estimate, with ties broken toward the smaller network. The 100 fits per training set (10 values of $P \times 10$ folds) ran in parallel with `joblib`.

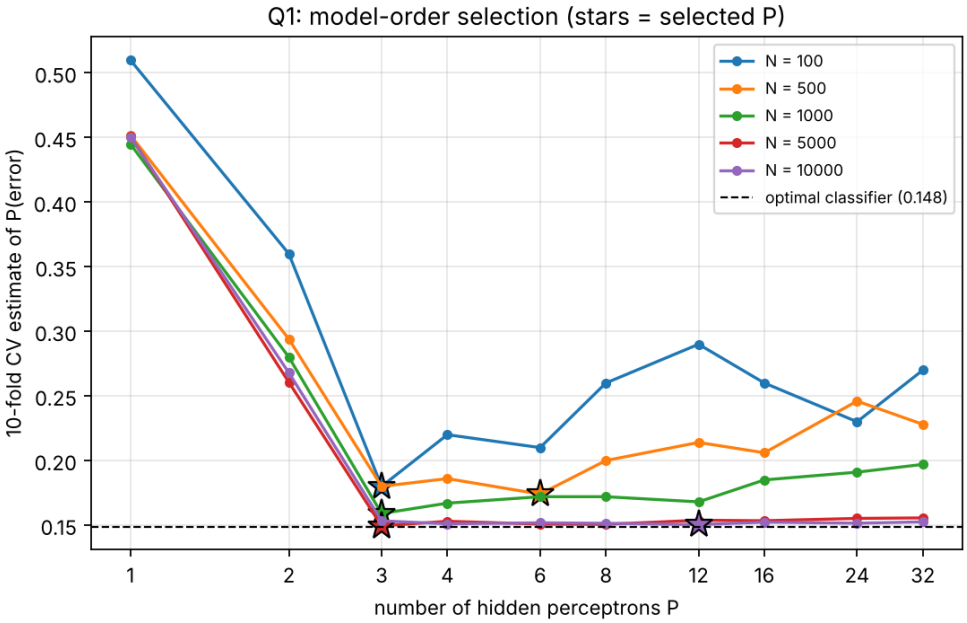


Figure 2: 10-fold CV estimate of P(error) versus P for each training set. Stars mark the selected P; the dashed line is the optimal classifier's test error.

$N \setminus P$	1	2	3	4	6	8	12	16	24	32
100	0.510	0.360	**0.180**	0.220	0.210	0.260	0.290	0.260	0.230	0.270
500	0.452	0.294	0.180	0.186	**0.174**	0.200	0.214	0.206	0.246	0.228
1000	0.445	0.280	**0.159**	0.167	0.172	0.172	0.168	0.185	0.191	0.197
5000	0.451	0.261	**0.149**	0.153	0.151	0.151	0.154	0.153	0.155	0.156
10000	0.451	0.268	0.153	0.151	0.152	0.151	**0.150**	0.152	0.151	0.152

Table 2: 10-fold CV estimates of P(error). Bold marks the selected P (minimum; ties go to the smaller P).

Final training and test-set performance

For each training set, an MLP with the selected P was retrained by ML on the *entire* training set with 10 random restarts, keeping the highest-likelihood solution. Each final model then classified the 100,000 test samples with the MAP rule.

N_{train}	P	#params	CV P(err.)	train P(err.)	**test P(err.)**	gap
100	3	28	0.180	0.0700	**0.2508**	+0.1024
500	6	52	0.174	0.1380	**0.1807**	+0.0323
1000	3	28	0.159	0.1520	**0.1523**	+0.0039
5000	3	28	0.149	0.1514	**0.1501**	+0.0017
10000	12	100	0.150	0.1482	**0.1496**	+0.0012
Optimal (true pdf)	–	–	–	–	**0.1484**	0

Table 3: Final MLPs, each trained by ML with 10 restarts on its full training set and evaluated on the 100,000-sample test set.

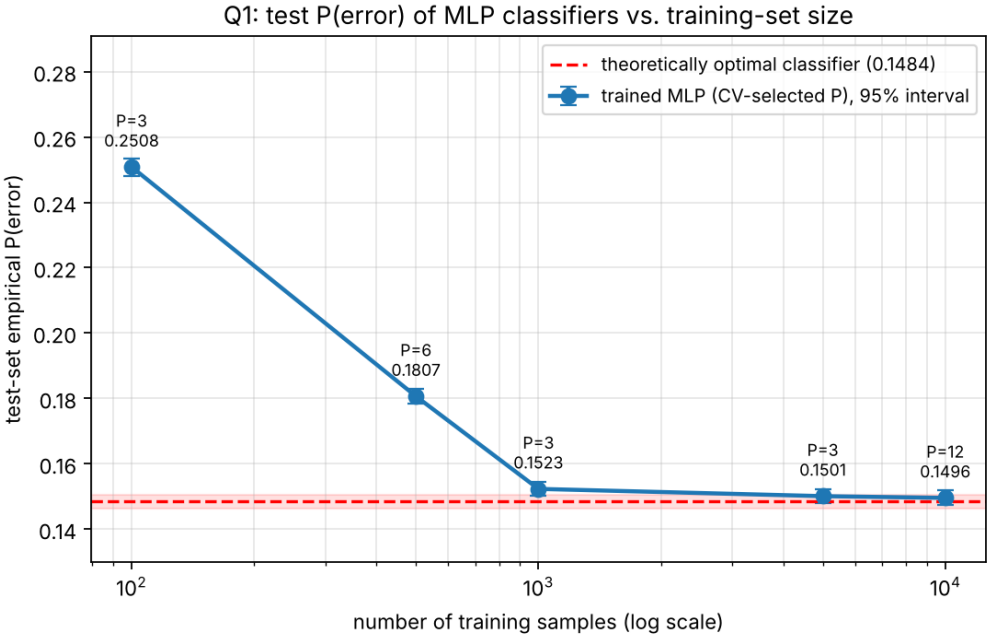


Figure 3: Test-set empirical P(error) of each trained MLP versus the number of training samples (semilog-x), with 95% Wilson binomial intervals. The red dashed line and band are the theoretically optimal classifier and its interval. The intervals reflect test-set sampling only, not variability over training sets or initializations; all classifiers share the same test set.

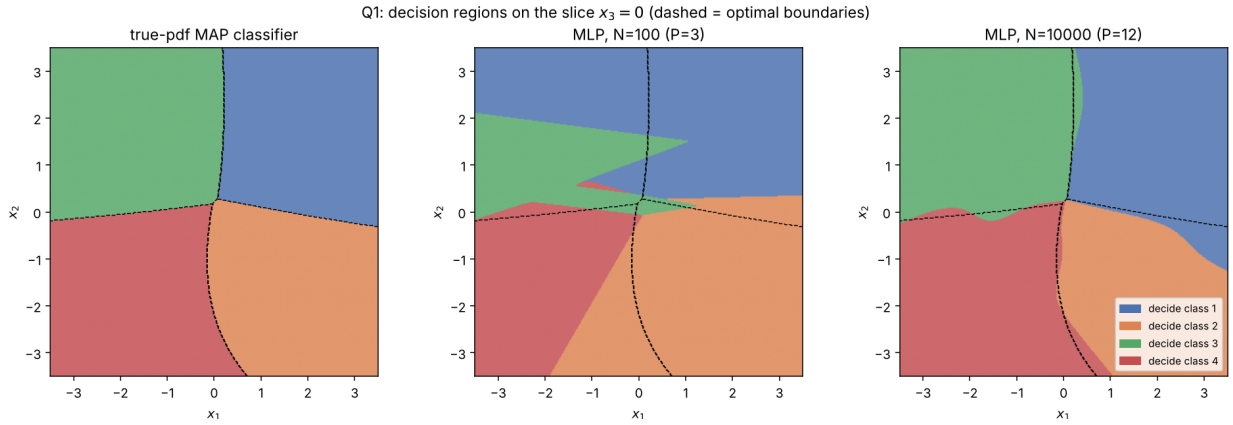


Figure 4: Decision regions on the plane $x_3 = 0$, a slice of the 3-D classifiers, for the true-pdf MAP rule, the $N = 100$ MLP and the $N = 10000$ MLP. Dashed lines are the optimal boundaries. The $N = 10000$ MLP reproduces the curved optimal boundaries closely, while the $N = 100$ model's regions are visibly distorted.

	$L = 1$	$L = 2$	$L = 3$	$L = 4$
$D = 1$	21566	1713	576	1520
$D = 2$	1143	20879	978	1470
$D = 3$	524	866	22293	1577
$D = 4$	1740	1694	1156	20305

Table 4: Test-set confusion counts of the $N = 10000$ MLP (rows = decision, columns = true label); compare with the optimal classifier's table above.

Discussion

- **Convergence to the optimum.** Test P(error) falls steadily with training-set size: 0.2508 at $N = 100$, 0.1807 at $N = 500$, 0.1523 at $N = 1000$, 0.1501 at $N = 5000$ and 0.1496 at $N = 10000$, against 0.1484 for the true-pdf MAP classifier. The 95% Wilson intervals (half-width ≈ 0.0022; see the figure) of the $N = 5000$ and $N = 10000$ models overlap the optimal classifier's interval. The trained MLPs therefore approach the Bayes decision rule, as expected of ML posterior estimates from a sufficiently flexible model.

- **What cross-validation selects.** Very small networks underfit badly for every N: $P = 1$ gives a CV error of about 0.45, because one hidden unit maps $\mathbb{R}^3$ onto a single scalar and cannot carve out four regions, and $P = 2$ gives about 0.27. From $P = 3$ upward the CV curve for large N is essentially flat at the Bayes level. Three smooth ramps are already enough to approximate the four quadratic-boundary regions well, so the extra capacity neither helps nor hurts once data are plentiful. For small N the curve is U-shaped: with 100 samples it rises from 0.18 at $P = 3$ to 0.26–0.29 at larger P. Unregularized ML training of a large network on few points fits noise. The $N = 100$ final model reaches a training error of 0.07 but a test error of 0.25.

- **Selected sizes need not grow monotonically with N.** The selected P is 3, 6, 3, 3, 12. For $N \geq 1000$ the CV errors of all $P \geq 3$ lie within about 0.005 of each other, comparable to the CV estimate's own sampling noise ($\sqrt{0.15 \cdot 0.85/N} \approx 0.004$–$0.011$). Which P wins is therefore

partly chance; what matters is that every candidate $P \geq 3$ performs at nearly the Bayes level. The $N = 10000$ choice of $P = 12$ (CV error 0.150, versus 0.153 for $P = 3$) is consistent with more data supporting a slightly richer model, and its test error is the lowest of the five.

- **Sources of the remaining gap.** At $N = 100$ and 500 the gap to optimal (0.10 and 0.03) is estimation error: too few samples to pin down 28–52 parameters by unpenalized ML. At $N \geq 5000$ the remaining gap of at most 0.0017 is within test-set sampling uncertainty. The training sets are independent draws rather than nested subsets, so the learning curve is not guaranteed to be monotone, though here it is.

How the software was verified

- **Gradient.** The MLP and its backpropagation were written directly in NumPy, so the gradient was checked against a central finite-difference approximation (`scipy.optimize.approx_fprime`) at a random parameter vector. The relative error was 4.4×10^{-7}, confirming that L-BFGS receives the exact gradient of the cross-entropy it minimizes.

- **Outputs.** The softmax outputs were checked to be positive and to sum to 1 for every sample. The log-softmax is computed by subtracting the row maximum, and softplus as `logaddexp(0,a)`, so neither overflows for large activations.

- **Objective.** The loss is exactly the average negative log-likelihood of the one-hot labels under the softmax outputs, i.e. the ML criterion. No weight decay or early stopping is applied, and every L-BFGS run starts from a new random initialization.

- **Cross-validation.** The code asserts that the 10 validation folds are disjoint from their training folds and together cover every training sample exactly once. Standardization statistics are recomputed inside each training call, so no validation data leak into training.

- **Test set.** The 100,000-sample test set is used only after model selection and final training, never for choosing P or for stopping.

- **Optimizer convergence.** The L-BFGS status of every restart was recorded by re-running the final fits and all cross-validation fits with their original seeds. The script asserts that every error count reproduces exactly. Here "converged" means the projected gradient fell below 10^{-7} before the iteration cap, a deliberately strict tolerance.

N	P	final: best restart converged?	final: restarts converged	CV: retained fits converged	CV: all restarts converged	final: training NLL range over restarts
100	3	yes (2636 it.)	8/10	86/100	171/200	[0.1625, 0.2148]
500	6	no (3000 it.)	0/10	18/100	36/200	[0.3653, 0.3837]
1000	3	yes (2004 it.)	10/10	20/100	39/200	[0.4048, 0.4085]
5000	3	yes (2837 it.)	10/10	15/100	33/200	[0.3874, 0.3962]
10000	12	no (3000 it.)	0/10	18/100	35/200	[0.3883, 0.3898]

Non-converged runs stopped at the iteration cap while still making very small improvements. They are kept at their achieved likelihood, never discarded. Non-convergence is rare at $N = 100$ (86% of retained CV fits converged) and common for $N \geq 500$ (15–20%). Unpenalized cross-entropy has very flat directions, along which weights can keep growing to sharpen posteriors that are already nearly right, so the strict gradient tolerance is hard to meet within the budget. The sensitivity check below tests whether this affects the selected sizes. The spread of final training NLL across the 10 restarts (last column) shows how much the restarts mattered.

Does non-convergence affect model selection? Because many CV fits stopped at the 1000-iteration cap, the CV for $N = 500$ and $N = 1000$ was repeated with a 5× larger budget of 5000 iterations, using the same seeds and folds (`q1_cv_sensitivity.py`). The selected sizes are identical: $P = 6$ for $N = 500$ and $P = 3$ for $N = 1000$, with 49% and 40% of the retained fits now converged. CV P(error) by P (bold = selected):

N	max. iterations	P=1	P=2	P=3	P=4	P=6	P=8	P=12	P=16	P=24	P=32
500	1000	0.452	0.294	0.180	0.186	**0.174**	0.200	0.214	0.206	0.246	0.228
500	5000	0.452	0.296	0.184	0.184	**0.176**	0.192	0.220	0.216	0.266	0.262
1000	1000	0.445	0.280	**0.159**	0.167	0.172	0.172	0.168	0.185	0.191	0.197
1000	5000	0.445	0.281	**0.161**	0.170	0.171	0.183	0.175	0.188	0.222	0.233

Running longer leaves small networks unchanged and makes large networks slightly *worse* on held-out data: they fit the training folds more closely. The iteration cap therefore acts as mild implicit regularization on oversized networks, but it does not change which P is selected.

- **End-to-end.** With 10,000 training samples, the MLP's test error comes within 0.12 percentage points of the true-pdf MAP classifier, as expected of a consistent posterior estimator with enough data and capacity.

Question 2: GMM model-order selection with cross-validation

True GMM

The true density is $p(\mathbf{x}) = \sum_{k=1}^{4} \alpha_k \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_k, \Sigma_k)$, $\mathbf{x} \in \mathbb{R}^2$, with

k	α_k	$\boldsymbol{\mu}_k$	Σ_k	mean eigenvalue of Σ_k
1	0.30	$[0, 0]^T$	$\begin{bmatrix} 1.0 & 0.3 \\ 0.3 & 0.8 \end{bmatrix}$	0.90
2	0.25	$[1.6, 0.6]^T$	$\begin{bmatrix} 0.7 & -0.2 \\ -0.2 & 1.0 \end{bmatrix}$	0.85
3	0.25	$[6, 4]^T$	$\begin{bmatrix} 1.5 & 0.6 \\ 0.6 & 0.9 \end{bmatrix}$	1.20
4	0.20	$[-5, 5]^T$	$\begin{bmatrix} 0.6 & 0 \\ 0 & 1.8 \end{bmatrix}$	1.20

All four components have different weights, means and covariances. Components 1 and 2 overlap strongly: their means are $\|\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2\| = 1.71$ apart, while the sum of their average covariance eigenvalues is 1.75, the same order as the question suggests. Together they form a single elongated lump. Components 3 and 4 are well separated from everything else.

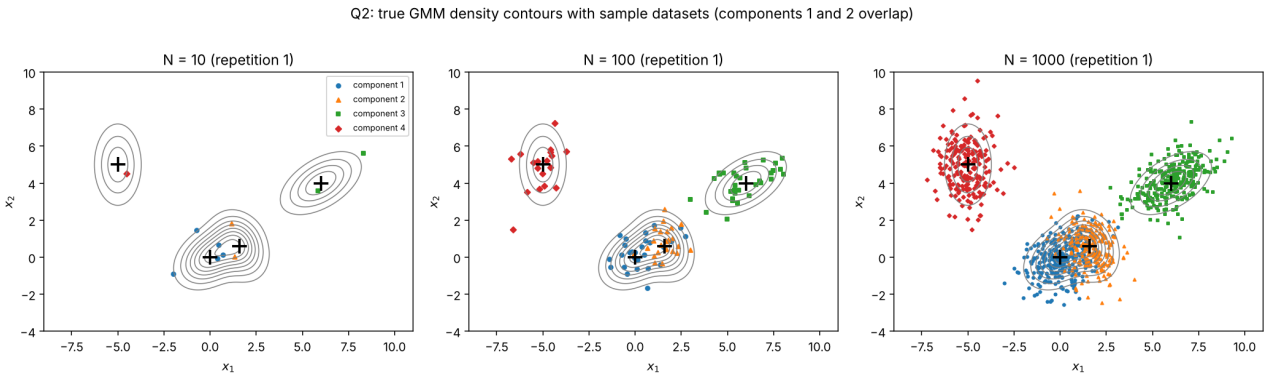


Figure 5: True GMM density contours with one dataset of each size, colored by the (hidden) generating component.

Procedure

For each repetition $r = 1, \ldots, R$ with $R = 100$, and each size $N \in \{10, 100, 1000\}$:

1. Draw a fresh iid dataset of N samples from the true GMM.
2. Shuffle it and split it into 10 disjoint folds. For $N = 10$ each fold holds one sample, so this is leave-one-out.
3. For each candidate order $K = 1, \ldots, 10$ and each fold, fit a K-component, full-covariance GMM to the other 9 folds by maximum likelihood with the EM algorithm (scikit-learn's `GaussianMixture`). Then compute the log-likelihood of the held-out fold under the fitted model, $\sum_{\mathbf{x} \in \text{fold}} \ln \hat{p}_K(\mathbf{x})$.
4. Sum the held-out log-likelihoods over the 10 folds; dividing by N gives the average validation log-likelihood $\overline{\ell}_K$. Every sample is scored exactly once by a model that did not see it.
5. Select $\hat{K} = \arg\max_K \overline{\ell}_K$.

The selection rate of order K at size N is the fraction of the R repetitions with $\hat{K} = K$. Both parameter estimation (EM maximizes training log-likelihood) and validation (held-out log-likelihood) use the log-likelihood of the data, as the question requires.

EM settings and why.

- *Local optima.* Each EM fit runs 3 k-means++ initializations (`n_init=3`) and keeps the highest-likelihood solution, with up to 500 iterations and tolerance 10^{-4}.
- *Covariance floor.* The GMM likelihood is unbounded: a component can collapse onto a single training point with covariance $\to 0$. To keep the ML problem well posed, $10^{-3} I$ is added to every covariance estimate (`reg_covar`). This is negligible compared with the true covariances, whose eigenvalues are 0.5–1.9. A collapsed component still wastes probability mass on held-out points, so CV still penalizes overfitting.
- *Infeasible orders.* A K-component GMM cannot be fitted to fewer than K points. For $N = 10$ each training split has 9 samples, so $K = 10$ is assigned validation log-likelihood $-\infty$ and can never be selected. This is a real limitation of 10 samples, not a software artifact.

Results

N	K=1	K=2	K=3	K=4	K=5	K=6	K=7	K=8	K=9	K=10	mean $\hat{K}$	median $\hat{K}$
10	**93**	6	1	0	0	0	0	0	0	0	1.08	1
100	0	0	**92**	8	0	0	0	0	0	0	3.08	3
1000	0	0	1	**83**	13	3	0	0	0	0	4.18	4

Table 5: Selection rate (%) of each GMM order over 100 independent repetitions (bold = most frequently selected). True order is $K = 4$.

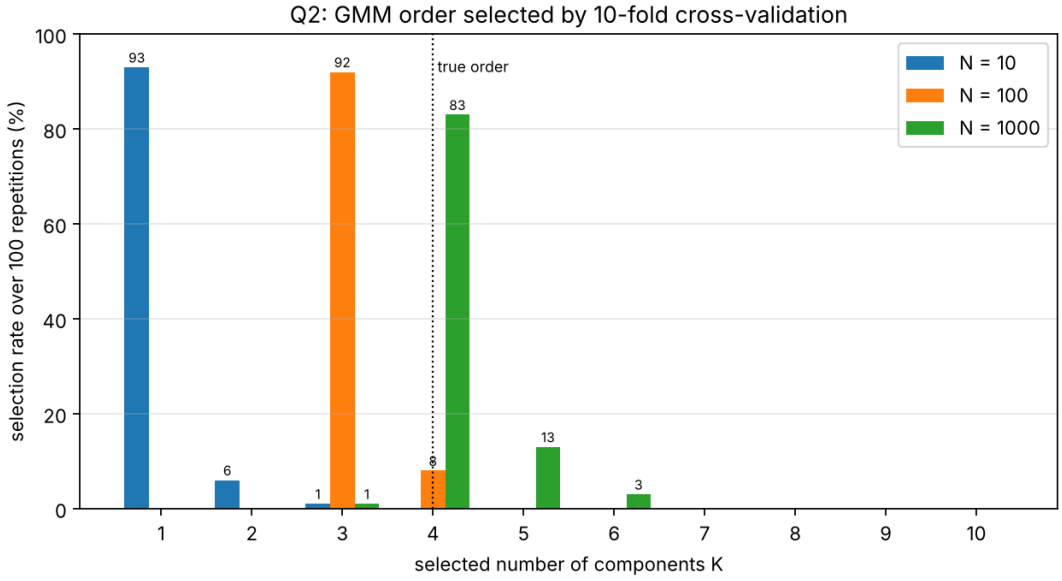


Figure 6: Selection rate of each GMM order over 100 repetitions, for each dataset size.

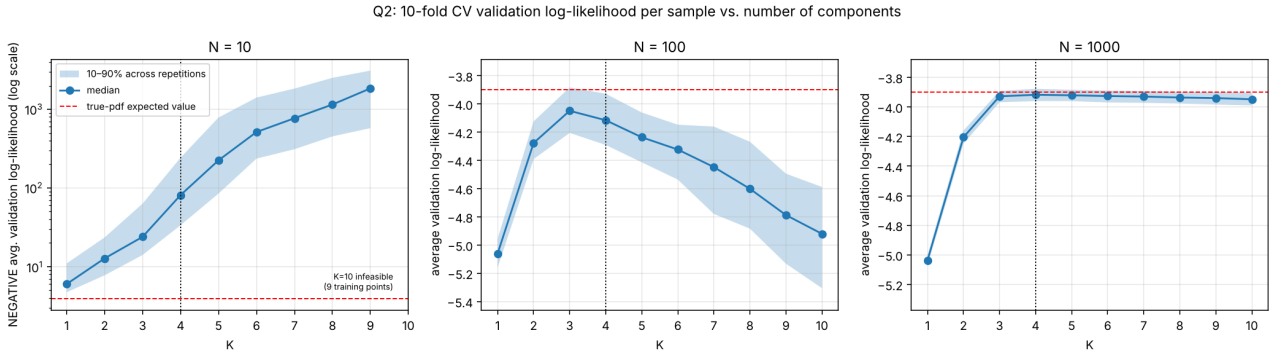


Figure 7: Average validation log-likelihood per sample versus K: median and 10–90% band across repetitions. The red line is $E[\ln p_{\text{true}}(\mathbf{x})] \approx -3.899$, estimated with 200,000 samples; it is the best value any model can achieve in expectation. For $N = 10$ the y-axis is clipped because large-K fits collapse onto single points and give very negative held-out scores.

Discussion

- $N = 10$**: cross-validation picks the simplest model (K=1 in 93% of repetitions).** Each training split has only nine points, and component 4 has on average only two samples in the whole dataset. A two-dimensional full-covariance Gaussian has 5 free parameters, so every extra component multiplies the estimation variance. Extra components typically lock onto one or two training points with tiny covariances, and the held-out point then falls far from them. The median validation log-likelihood falls from -6.0 at $K = 1$ to -81 at $K = 4$ and below -1000 at $K \geq 8$, more than two orders of magnitude worse than the true density's -3.90. A single broad Gaussian covering all three clusters is the best predictive model this little data can support. Ten samples carry essentially no information about the true order.

- $N = 100$**: the two overlapping components are merged (K=3 in 92%).** With 100 samples the three *visible* clusters are well estimated, which explains the jump in validation log-likelihood from $K = 2$ to $K = 3$. Components 1 and 2, however, have means only 1.71 apart, comparable to the sum of their average eigenvalues ($0.90 + 0.85 = 1.75$). Their combined density is a single unimodal, slightly elongated lump, and roughly 55 points in it are not enough to estimate the 6 extra parameters a split requires. In 92 of 100 repetitions the 4-component model scored lower on held-out data than the 3-component model; the median difference was -0.065 nats per sample. The bias of merging the pair costs less than the variance of separating it.

- $N = 1000$**: the true order dominates (K=4 in 83%).** With roughly 550 points in the overlapping pair, the 4-component model's better fit outweighs its extra variance. $K = 4$ beat $K = 3$ on held-out likelihood in 97% of repetitions, by a median of 0.013 nats per sample, and the median CV curve peaks at $K = 4$ (-3.918), within 0.02 of the true-density value -3.899. The remaining selections mostly lean toward *over*fitting ($K = 5$: 13%, $K = 6$: 3%). The validation curve beyond $K = 4$ is very flat (median drop of only 0.004 from $K = 4$ to 5), because a superfluous component can sit almost redundantly inside an existing one at little cost. With that little separation, sampling noise sometimes tips the choice upward. Underfitting ($K = 3$) occurs only once.

- **Overall pattern.** The selected order rises with the amount of data (mean $\hat{K}$: 1.08, 3.08, 4.18). Cross-validated log-likelihood estimates the model with the best *predictive* density for the data at hand, not the number of generating components. With limited data it deliberately prefers simpler models, and it recovers the true structure only once the data can resolve it. The overlapping pair is exactly where the true order is hardest to identify, and it accounts for nearly all the $K = 3$ choices at $N = 100$.

- **Statistical precision.** Each rate is a proportion out of $R = 100$ independent repetitions, with standard error $\sqrt{r(1-r)/100} \leq 0.05$. For example, the $N = 1000$ rate for $K = 4$ is 0.83 ± 0.04. The ranking of the most frequent order at every N is therefore well beyond Monte Carlo noise. Rates of 0% mean "not observed in 100 trials", not "impossible".

- **Sensitivity to settings.** The covariance floor ($10^{-3}I$) and the 3 EM restarts are part of the fitted model family. A larger floor would soften the large-K collapse at $N = 10$; the conclusions at $N = 100$ and $N = 1000$ come from well-populated components, whose covariance eigenvalues are hundreds of times larger than the floor, so they are not driven by it.

Acknowledgement of resources and AI use

[**EDIT BEFORE SUBMITTING: make this statement accurately describe how you used these tools, then delete this line.**] I used Claude (Anthropic, model Claude Opus 5.5, via claude.ai, November 2026) as a coding and writing assistant for this assignment. It helped design the data distributions, write the Python implementations in the Appendix, run the experiments, produce the figures, and draft and typeset this report. I reviewed the procedures and results. Libraries used: NumPy, SciPy (`scipy.optimize.minimize`, `scipy.stats.multivariate_normal`), scikit-learn (`KFold`, `GaussianMixture`), joblib, and Matplotlib.

Appendix: Code

mlp.py

```
"""Minimal 2-layer MLP for C-class posterior estimation, written in NumPy.

Architecture:  x (d) -> hidden layer of P softplus perceptrons -> C-way softmax.
Training:      maximum likelihood = minimum average cross-entropy, full-batch
               L-BFGS (scipy.optimize.minimize) with an analytic gradient.
"""
import numpy as np
from scipy.optimize import minimize


def softplus(a):
    # smooth ReLU: log(1 + e^a), computed stably
    return np.logaddexp(0.0, a)


def sigmoid(a):
    return 0.5 * (1.0 + np.tanh(0.5 * a))      # derivative of softplus, overflow-safe


def log_softmax(A):
    A = A - A.max(axis=1, keepdims=True)
    return A - np.log(np.exp(A).sum(axis=1, keepdims=True))


class MLP:
    def __init__(self, d, P, C):
        self.d, self.P, self.C = d, P, C
        self.shapes = [(P, d), (P,), (C, P), (C,)]
        self.sizes = [int(np.prod(s)) for s in self.shapes]
        self.n_params = sum(self.sizes)
        self.theta = None
        self.mu = np.zeros(d)
        self.sd = np.ones(d)

    # ---- parameter packing ----
    def unpack(self, theta):
        out, k = [], 0
        for s, n in zip(self.shapes, self.sizes):
            out.append(theta[k:k + n].reshape(s)); k += n
        return out

    def init(self, rng):
        # Glorot-style random initialization (a new draw for every restart)
        W1 = rng.normal(0, np.sqrt(2.0 / (self.d + self.P)), (self.P, self.d))
        b1 = rng.normal(0, 0.5, self.P)
        W2 = rng.normal(0, np.sqrt(2.0 / (self.P + self.C)), (self.C, self.P))
        b2 = np.zeros(self.C)
        return np.concatenate([W1.ravel(), b1, W2.ravel(), b2])

    # ---- forward pass ----
    def _forward(self, theta, X):
        W1, b1, W2, b2 = self.unpack(theta)
        A1 = X @ W1.T + b1               # N x P pre-activations
        H = softplus(A1)                 # N x P hidden outputs
        A2 = H @ W2.T + b2               # N x C logits
        return A1, H, log_softmax(A2)

    # ---- objective: average negative log-likelihood and its gradient ----
    def nll_grad(self, theta, X, Y):
        """X: N x d (standardized), Y: N x C one-hot."""
        W1, b1, W2, b2 = self.unpack(theta)
        A1, H, logp = self._forward(theta, X)
        N = X.shape[0]
        nll = -np.sum(Y * logp) / N
```

```
        G2 = (np.exp(logp) - Y) / N      # dNLL/dA2 for softmax + cross-entropy
        gW2 = G2.T @ H
        gb2 = G2.sum(axis=0)
        G1 = (G2 @ W2) * sigmoid(A1)     # back through softplus
        gW1 = G1.T @ X
        gb1 = G1.sum(axis=0)
        return nll, np.concatenate([gW1.ravel(), gb1, gW2.ravel(), gb2])

    # ---- public API ----
    def fit(self, X, labels, restarts=5, rng=None, maxiter=3000):
        """ML training with `restarts` random initializations; keeps the solution
        with the highest training log-likelihood (lowest average NLL)."""
        rng = np.random.default_rng() if rng is None else rng
        self.mu, self.sd = X.mean(axis=0), X.std(axis=0) + 1e-12   # standardize with training stats
        Xs = (X - self.mu) / self.sd
        Y = np.eye(self.C)[labels]
        best = None
        self.restart_log = []            # (final NLL, converged?, iterations) for every restart
        for _ in range(restarts):
            res = minimize(self.nll_grad, self.init(rng), args=(Xs, Y), jac=True,
                           method="L-BFGS-B", options={"maxiter": maxiter, "gtol": 1e-7})
            self.restart_log.append((float(res.fun), bool(res.success), int(res.nit)))
            if best is None or res.fun < best.fun:
                best = res
        self.theta = best.x
        self.train_nll = best.fun
        self.converged = bool(best.success)
        self.n_iter = int(best.nit)
        return self

    def predict_proba(self, X):
        _, _, logp = self._forward(self.theta, (X - self.mu) / self.sd)
        return np.exp(logp)

    def predict(self, X):
        # MAP rule with the MLP posterior estimates (0-1 loss -> argmax posterior)
        _, _, logp = self._forward(self.theta, (X - self.mu) / self.sd)
        return logp.argmax(axis=1)
```

q1.py

```
"""EECE5644 Assignment 3 - Question 1
4-class, 3-D Gaussian data; theoretically optimal MAP classifier; 2-layer MLPs whose
hidden-layer size P is chosen by 10-fold cross-validation (min P(error)), trained by
maximum likelihood with random restarts, and evaluated on a 100,000-sample test set.
"""
import os
os.environ.setdefault("OMP_NUM_THREADS", "1")
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
import json, time
import numpy as np
from scipy.stats import multivariate_normal as mvn
from scipy.optimize import approx_fprime
from sklearn.model_selection import KFold
from joblib import Parallel, delayed
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from mlp import MLP

os.makedirs("figs", exist_ok=True)
SEED = 5644
rng = np.random.default_rng(SEED)
C, d = 4, 3

# ---------------- true data distribution ----------------
def rot(a, b, c):
```

```
    Rx = np.array([[1, 0, 0], [0, np.cos(a), -np.sin(a)], [0, np.sin(a), np.cos(a)]])
    Ry = np.array([[np.cos(b), 0, np.sin(b)], [0, 1, 0], [-np.sin(b), 0, np.cos(b)]])
    Rz = np.array([[np.cos(c), -np.sin(c), 0], [np.sin(c), np.cos(c), 0], [0, 0, 1]])
    return Rz @ Ry @ Rx

scale = 0.85                                  # tuned so Bayes error is ~15%
means = scale * np.array([[1, 1, 1], [1, -1, -1], [-1, 1, -1], [-1, -1, 1]], float)
eigs = [[1.0, 0.6, 0.3], [0.8, 0.8, 0.4], [1.2, 0.5, 0.5], [0.7, 0.6, 0.9]]
angs = [(0.3, 0.5, 0.2), (1.0, -0.4, 0.8), (-0.6, 0.9, -0.3), (0.2, 0.2, 1.2)]
covs = [rot(*a) @ np.diag(e) @ rot(*a).T for a, e in zip(angs, eigs)]
priors = np.full(C, 1 / C)


def generate(N):
    L = rng.choice(C, size=N, p=priors)
    X = np.zeros((N, d))
    for c in range(C):
        i = L == c
        X[i] = rng.multivariate_normal(means[c], covs[c], i.sum())
    return X, L


def map_classify(X):
    # min-P(error) rule: argmax_c  ln p(x|c) + ln P(c)
    return np.column_stack([mvn.logpdf(X, means[c], covs[c]) + np.log(priors[c])
                            for c in range(C)]).argmax(axis=1)


Ns = [100, 500, 1000, 5000, 10000]
train = {N: generate(N) for N in Ns}
Xtest, Ltest = generate(100_000)

# ---------------- theoretically optimal classifier ----------------
D_opt = map_classify(Xtest)
pe_opt = float(np.mean(D_opt != Ltest))
conf_opt = np.array([[np.sum((D_opt == i) & (Ltest == j)) for j in range(C)] for i in range(C)])
print(f"Test class counts: {np.bincount(Ltest, minlength=C).tolist()}")
print(f"Optimal (true-pdf MAP) classifier test P(error) = {pe_opt:.4f}")
print("Confusion counts (rows = decision, cols = true label):\n", conf_opt)

# ---------------- tool sanity checks ----------------
chk = MLP(d, 5, C); r0 = np.random.default_rng(0)
Xc, Yc = r0.normal(size=(40, d)), np.eye(C)[r0.integers(0, C, 40)]
th = chk.init(r0)
num = approx_fprime(th, lambda t: chk.nll_grad(t, Xc, Yc)[0], 1e-6)
ana = chk.nll_grad(th, Xc, Yc)[1]
grad_rel_err = float(np.linalg.norm(num - ana) / np.linalg.norm(ana))
print(f"Gradient check: relative error between analytic and finite-difference gradient = {grad_rel_err:.2e}")
chk.theta = th; chk.mu, chk.sd = np.zeros(d), np.ones(d)
print("Softmax outputs sum to 1:", np.allclose(chk.predict_proba(Xc).sum(axis=1), 1.0))
# a sufficiently large MLP trained on lots of data should approach the Bayes classifier
# (checked at the end via the N=10000 result)

# ---------------- model-order selection by 10-fold CV ----------------
P_grid = [1, 2, 3, 4, 6, 8, 12, 16, 24, 32]
K = 10
CV_RESTARTS, CV_MAXITER = 2, 1000
FINAL_RESTARTS, FINAL_MAXITER = 10, 3000


def cv_job(X, L, P, tr, va, seed):
    m = MLP(d, P, C).fit(X[tr], L[tr], restarts=CV_RESTARTS,
                         rng=np.random.default_rng(seed), maxiter=CV_MAXITER)
    return int(np.sum(m.predict(X[va]) != L[va]))


results = {"pe_opt": pe_opt, "conf_opt": conf_opt.tolist(), "P_grid": P_grid,
           "grad_rel_err": grad_rel_err, "sets": {}}
t0 = time.time()
```

```
for N in Ns:
    X, L = train[N]
    folds = list(KFold(n_splits=K, shuffle=True, random_state=SEED).split(X))
    # check the folds partition the data: disjoint validation sets that cover every sample once
    allva = np.concatenate([va for _, va in folds])
    assert np.array_equal(np.sort(allva), np.arange(N))
    assert all(len(np.intersect1d(tr, va)) == 0 for tr, va in folds)
    jobs = [(P, f) for P in P_grid for f in range(K)]
    errs = Parallel(n_jobs=2)(delayed(cv_job)(X, L, P, *folds[f], SEED + 1000 * P + f)
                              for P, f in jobs)
    cv_err = {P: sum(e for (p, _), e in zip(jobs, errs) if p == P) / N for P in P_grid}
    P_best = min(P_grid, key=lambda P: (cv_err[P], P))      # ties -> smaller network
    # final ML training on the whole training set with many restarts
    final = MLP(d, P_best, C).fit(X, L, restarts=FINAL_RESTARTS,
                                  rng=np.random.default_rng(SEED + N), maxiter=FINAL_MAXITER)
    D = final.predict(Xtest)
    pe_test = float(np.mean(D != Ltest))
    pe_train = float(np.mean(final.predict(X) != L))
    results["sets"][N] = {"cv_err": cv_err, "P_best": P_best, "pe_test": pe_test,
                          "pe_train": pe_train, "train_nll": float(final.train_nll),
                          "n_params": final.n_params}
    print(f"N={N:6d}: CV P(error) by P = " + ", ".join(f"{P}:{cv_err[P]:.3f}" for P in P_grid))
    print(f"          best P = {P_best} ({final.n_params} params), train P(err) = {pe_train:.4f}, "
          f"train NLL = {final.train_nll:.4f}, TEST P(error) = {pe_test:.4f}   [{time.time()-t0:.0f}s]",
          flush=True)

with open("q1_results.json", "w") as f:
    json.dump(results, f, indent=1)

# ---------------- figures ----------------
fig = plt.figure(figsize=(6.5, 5.5))
ax = fig.add_subplot(projection="3d")
X, L = train[1000]
for c, mk in zip(range(C), "o^sD"):
    ax.scatter(*X[L == c].T, s=6, marker=mk, alpha=0.6, label=f"class {c+1}")
ax.set_xlabel("$x_1$"); ax.set_ylabel("$x_2$"); ax.set_zlabel("$x_3$")
ax.set_title("Q1: training set with N = 1000"); ax.legend(fontsize=8)
fig.tight_layout(); fig.savefig("figs/q1_data.png", dpi=150); plt.close(fig)

fig, ax = plt.subplots(figsize=(7, 4.6))
for N in Ns:
    ce = results["sets"][N]["cv_err"]
    line, = ax.plot(P_grid, [ce[P] for P in P_grid], "o-", ms=4, label=f"N = {N}")
    Pb = results["sets"][N]["P_best"]
    ax.plot(Pb, ce[Pb], "*", ms=14, color=line.get_color(), mec="k")
ax.axhline(pe_opt, color="k", ls="--", lw=1, label=f"optimal classifier ({pe_opt:.3f})")
ax.set_xscale("log", base=2); ax.set_xticks(P_grid, [str(p) for p in P_grid])
ax.set_xlabel("number of hidden perceptrons P"); ax.set_ylabel("10-fold CV estimate of P(error)")
ax.set_title("Q1: model-order selection (stars = selected P)")
ax.legend(fontsize=8); ax.grid(alpha=0.3)
fig.tight_layout(); fig.savefig("figs/q1_cv.png", dpi=160); plt.close(fig)

fig, ax = plt.subplots(figsize=(7, 4.6))
pe = [results["sets"][N]["pe_test"] for N in Ns]
ax.semilogx(Ns, pe, "o-", lw=2, ms=7, label="trained MLP (CV-selected P)")
for N, p in zip(Ns, pe):
    ax.annotate(f"P={results['sets'][N]['P_best']}\n{p:.4f}", (N, p), textcoords="offset points",
                xytext=(0, 9), ha="center", fontsize=8)
ax.axhline(pe_opt, color="r", ls="--", label=f"theoretically optimal classifier ({pe_opt:.4f})")
ax.set_xlabel("number of training samples (log scale)"); ax.set_ylabel("test-set empirical P(error)")
ax.set_title("Q1: test P(error) of MLP classifiers vs. training-set size")
ax.set_ylim(min(pe_opt, min(pe)) - 0.02, max(pe) + 0.05)
ax.legend(fontsize=9); ax.grid(alpha=0.3, which="both")
fig.tight_layout(); fig.savefig("figs/q1_test_perror.png", dpi=160); plt.close(fig)
print(f"done in {time.time()-t0:.0f}s")
```

q1_diagnostics.py

```
"""EECE5644 Assignment 3 - Question 1 diagnostics (run after q1.py).
Re-creates the exact datasets and final MLPs of q1.py (same seeds) and records:
  * optimizer convergence of every restart of every final fit,
  * convergence of the cross-validation fits (re-run with their original seeds),
  * 95% Wilson intervals for each test P(error),
  * the test confusion matrix of the N=10000 MLP,
  * a decision-region slice at x3 = 0 (true-pdf MAP vs N=10000 MLP).
Imports q1's data definitions without re-running its experiment.
"""
import os
os.environ.setdefault("OMP_NUM_THREADS", "1")
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
import json
import numpy as np
from scipy.stats import multivariate_normal as mvn, norm
from sklearn.model_selection import KFold
from joblib import Parallel, delayed
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from mlp import MLP

# ---- reproduce q1.py's random stream exactly (same generator, same draw order) ----
SEED = 5644
rng = np.random.default_rng(SEED)
C, d = 4, 3
exec(open("q1.py").read().split("# ---------------- true data distribution ----------------")[1]
     .split("Ns = [100")[0])                       # rot, means, covs, priors, generate, map_classify
Ns = [100, 500, 1000, 5000, 10000]
train = {N: generate(N) for N in Ns}
Xtest, Ltest = generate(100_000)
res1 = json.load(open("q1_results.json"))
assert abs(np.mean(map_classify(Xtest) != Ltest) - res1["pe_opt"]) < 1e-12, "data stream mismatch"
P_grid = res1["P_grid"]


def wilson(k, n, z=norm.ppf(0.975)):
    p = k / n
    c = (p + z * z / (2 * n)) / (1 + z * z / n)
    h = z * np.sqrt(p * (1 - p) / n + z * z / (4 * n * n)) / (1 + z * z / n)
    return c - h, c + h


def cv_status(X, L, P, tr, va, seed):
    m = MLP(d, P, C).fit(X[tr], L[tr], restarts=2, rng=np.random.default_rng(seed), maxiter=1000)
    return int(np.sum(m.predict(X[va]) != L[va])), m.converged, [r[1] for r in m.restart_log]


out = {"final": {}, "cv": {}}
n_test = len(Ltest)
k_opt = int(np.sum(map_classify(Xtest) != Ltest))
out["opt_ci"] = wilson(k_opt, n_test)
models = {}
for N in Ns:
    X, L = train[N]
    Pb = res1["sets"][str(N)]["P_best"]
    m = MLP(d, Pb, C).fit(X, L, restarts=10, rng=np.random.default_rng(SEED + N), maxiter=3000)
    k = int(np.sum(m.predict(Xtest) != Ltest))
    assert abs(k / n_test - res1["sets"][str(N)]["pe_test"]) < 1e-12, f"final model mismatch N={N}"
    models[N] = m
    out["final"][N] = {"P": Pb, "test_errors": k, "ci": wilson(k, n_test),
                       "best_converged": m.converged, "best_iters": m.n_iter,
                       "restarts_converged": int(sum(r[1] for r in m.restart_log)),
                       "restart_nll_min": min(r[0] for r in m.restart_log),
                       "restart_nll_max": max(r[0] for r in m.restart_log)}
    print(f"N={N}: P={Pb} test errors={k} 95% CI=({out['final'][N]['ci'][0]:.4f},{out['final'][N]['ci'][1]:.4f}) "
          f"best restart converged={m.converged} ({m.n_iter} it); converged restarts "
```

```
          f"{out['final'][N]['restarts_converged']}/10; restart NLL range "
          f"[{out['final'][N]['restart_nll_min']:.4f}, {out['final'][N]['restart_nll_max']:.4f}]", flush=True)

    # CV convergence (same seeds as q1.py, so the error counts must reproduce)
    folds = list(KFold(n_splits=10, shuffle=True, random_state=SEED).split(X))
    jobs = [(P, f) for P in P_grid for f in range(10)]
    st = Parallel(n_jobs=2)(delayed(cv_status)(X, L, P, *folds[f], SEED + 1000 * P + f) for P, f in jobs)
    for P in P_grid:
        errs = sum(s[0] for (p, _), s in zip(jobs, st) if p == P)
        assert abs(errs / N - res1["sets"][str(N)]["cv_err"][str(P)]) < 1e-12, f"CV mismatch N={N} P={P}"
    retained_ok = sum(s[1] for s in st)
    all_ok = sum(sum(s[2]) for s in st)
    out["cv"][N] = {"retained_converged": retained_ok, "retained_total": len(st),
                    "runs_converged": all_ok, "runs_total": 2 * len(st)}
    print(f"   CV fits: retained solution converged {retained_ok}/{len(st)}; "
          f"all restarts converged {all_ok}/{2*len(st)}", flush=True)

conf = np.array([[np.sum((models[10000].predict(Xtest) == i) & (Ltest == j)) for j in range(C)] for i in range(C)])
out["conf_mlp10k"] = conf.tolist()
print("N=10000 MLP confusion (rows decision, cols label):\n", conf)
json.dump(out, open("q1_diagnostics.json", "w"), indent=1, default=float)

# decision-region slice at x3 = 0
g = np.linspace(-3.5, 3.5, 351)
G1, G2 = np.meshgrid(g, g)
Gx = np.column_stack([G1.ravel(), G2.ravel(), np.zeros(G1.size)])
fig, axes = plt.subplots(1, 3, figsize=(14, 4.6))
cmap = matplotlib.colors.ListedColormap(["#4C72B0", "#DD8452", "#55A868", "#C44E52"])
for ax, (title, D) in zip(axes, [("true-pdf MAP classifier", map_classify(Gx)),
                                 (f"MLP, N=100 (P={models[100].P})", models[100].predict(Gx)),
                                 (f"MLP, N=10000 (P={models[10000].P})", models[10000].predict(Gx))]):
    ax.imshow(D.reshape(G1.shape), origin="lower", extent=[g[0], g[-1], g[0], g[-1]], cmap=cmap,
              vmin=-0.5, vmax=3.5, alpha=0.85)
    ax.contour(G1, G2, map_classify(Gx).reshape(G1.shape), levels=[0.5, 1.5, 2.5], colors="k",
               linewidths=0.8, linestyles="--")
    ax.set_title(title); ax.set_xlabel("$x_1$"); ax.set_ylabel("$x_2$")
handles = [matplotlib.patches.Patch(color=cmap(c), label=f"decide class {c+1}") for c in range(C)]
axes[2].legend(handles=handles, fontsize=8, loc="lower right")
fig.suptitle("Q1: decision regions on the slice $x_3=0$ (dashed = optimal boundaries)")
fig.tight_layout(); fig.savefig("figs/q1_decision_slice.png", dpi=140); plt.close(fig)

# test P(error) with Wilson intervals (replaces the plain plot)
fig, ax = plt.subplots(figsize=(7, 4.6))
pe = np.array([out["final"][N]["test_errors"] / n_test for N in Ns])
lo = np.array([out["final"][N]["ci"][0] for N in Ns]); hi = np.array([out["final"][N]["ci"][1] for N in Ns])
ax.errorbar(Ns, pe, yerr=[pe - lo, hi - pe], fmt="o-", lw=2, ms=7, capsize=4,
            label="trained MLP (CV-selected P), 95% interval")
for N, p in zip(Ns, pe):
    ax.annotate(f"P={out['final'][N]['P']}\n{p:.4f}", (N, p), textcoords="offset points",
                xytext=(0, 10), ha="center", fontsize=8)
ax.axhline(res1["pe_opt"], color="r", ls="--", label=f"theoretically optimal classifier ({res1['pe_opt']:.4f})")
ax.axhspan(*out["opt_ci"], color="r", alpha=0.12)
ax.set_xscale("log")
ax.set_xlabel("number of training samples (log scale)"); ax.set_ylabel("test-set empirical P(error)")
ax.set_title("Q1: test P(error) of MLP classifiers vs. training-set size")
ax.set_ylim(0.13, pe.max() + 0.04); ax.legend(fontsize=9); ax.grid(alpha=0.3, which="both")
fig.tight_layout(); fig.savefig("figs/q1_test_perror.png", dpi=160); plt.close(fig)
print("diagnostics done")
```

q1__cv__sensitivity.py

```
"""EECE5644 Assignment 3 - Q1 sensitivity check (run after q1.py).
Re-runs the 10-fold CV for N=500 and N=1000 with a 5x larger L-BFGS iteration budget
(5000 instead of 1000) to test whether non-converged CV fits change the selected P.
"""
import os
os.environ.setdefault("OMP_NUM_THREADS", "1")
```

```
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
import json
import numpy as np
from scipy.stats import multivariate_normal as mvn
from sklearn.model_selection import KFold
from joblib import Parallel, delayed
from mlp import MLP

SEED = 5644
rng = np.random.default_rng(SEED)
C, d = 4, 3
exec(open("q1.py").read().split("# ---------------- true data distribution ----------------")[1]
     .split("Ns = [100")[0])
Ns = [100, 500, 1000, 5000, 10000]
train = {N: generate(N) for N in Ns}
res1 = json.load(open("q1_results.json"))
P_grid = res1["P_grid"]


def job(X, L, P, tr, va, seed):
    m = MLP(d, P, C).fit(X[tr], L[tr], restarts=2, rng=np.random.default_rng(seed), maxiter=5000)
    return int(np.sum(m.predict(X[va]) != L[va])), m.converged


out = {}
for N in (500, 1000):
    X, L = train[N]
    folds = list(KFold(n_splits=10, shuffle=True, random_state=SEED).split(X))
    jobs = [(P, f) for P in P_grid for f in range(10)]
    r = Parallel(n_jobs=2)(delayed(job)(X, L, P, *folds[f], SEED + 1000 * P + f) for P, f in jobs)
    cv = {P: sum(e for (p, _), (e, _) in zip(jobs, r) if p == P) / N for P in P_grid}
    conv = sum(c for _, c in r)
    Pb = min(P_grid, key=lambda P: (cv[P], P))
    orig = res1["sets"][str(N)]
    out[N] = {"cv_err_5000": cv, "P_best_5000": Pb, "converged": conv, "total": len(r)}
    print(f"N={N}: maxiter 1000 -> P={orig['P_best']}; maxiter 5000 -> P={Pb}; converged {conv}/{len(r)}")
    print("   P:        " + " ".join(f"{P:>6}" for P in P_grid))
    print("   it=1000:  " + " ".join(f"{orig['cv_err'][str(P)]:6.3f}" for P in P_grid))
    print("   it=5000:  " + " ".join(f"{cv[P]:6.3f}" for P in P_grid), flush=True)
json.dump(out, open("q1_cv_sensitivity.json", "w"), indent=1)
```

q2.py

```
"""EECE5644 Assignment 3 - Question 2
GMM model-order selection with 10-fold cross-validation (validation log-likelihood),
for datasets of 10, 100 and 1000 samples drawn from a 4-component 2-D GMM in which two
components overlap strongly. Repeated R times to estimate the selection rate of each order.
"""
import os
os.environ.setdefault("OMP_NUM_THREADS", "1")
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
import json, sys, time, warnings
import numpy as np
from sklearn.mixture import GaussianMixture
from sklearn.model_selection import KFold
from sklearn.exceptions import ConvergenceWarning
from scipy.stats import multivariate_normal as mvn
from joblib import Parallel, delayed
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

os.makedirs("figs", exist_ok=True)
SEED = 5644
R = int(sys.argv[1]) if len(sys.argv) > 1 and sys.argv[1].isdigit() else 100        # repetitions
Ns = [10, 100, 1000]
Ks = list(range(1, 11))
```

```
NFOLD = 10
REG_COVAR = 1e-3        # covariance floor added to each diagonal (see report)
N_INIT = 3              # EM restarts per fit (k-means++ initializations)

# ---------------- true GMM ----------------
w_true = np.array([0.30, 0.25, 0.25, 0.20])
m_true = np.array([[0.0, 0.0], [1.6, 0.6], [6.0, 4.0], [-5.0, 5.0]])
C_true = np.array([[[1.0, 0.3], [0.3, 0.8]],
                   [[0.7, -0.2], [-0.2, 1.0]],
                   [[1.5, 0.6], [0.6, 0.9]],
                   [[0.6, 0.0], [0.0, 1.8]]])
avg_eig = [np.linalg.eigvalsh(C).mean() for C in C_true]
dist12 = np.linalg.norm(m_true[0] - m_true[1])


def sample_gmm(N, rng):
    comp = rng.choice(4, size=N, p=w_true)
    X = np.zeros((N, 2))
    for k in range(4):
        i = comp == k
        X[i] = rng.multivariate_normal(m_true[k], C_true[k], i.sum())
    return X, comp


def true_logpdf(X):
    return np.logaddexp.reduce(np.column_stack(
        [np.log(w_true[k]) + mvn.logpdf(X, m_true[k], C_true[k]) for k in range(4)]), axis=1)


# ---------------- one repetition ----------------
def one_run(N, rep):
    rng = np.random.default_rng([SEED, N, rep])
    X, _ = sample_gmm(N, rng)
    folds = KFold(n_splits=NFOLD, shuffle=True, random_state=int(rng.integers(1 << 31))).split(X)
    val_ll = np.zeros(len(Ks))
    for tr, va in folds:
        for j, K in enumerate(Ks):
            if K > len(tr):            # cannot fit more components than training samples
                val_ll[j] = -np.inf
                continue
            gm = GaussianMixture(n_components=K, covariance_type="full", reg_covar=REG_COVAR,
                                 n_init=N_INIT, init_params="k-means++", max_iter=500,
                                 tol=1e-4, random_state=int(rng.integers(1 << 31)))
            with warnings.catch_warnings():
                warnings.simplefilter("ignore", ConvergenceWarning)
                gm.fit(X[tr])                                  # ML (EM) on training folds
            val_ll[j] += gm.score_samples(X[va]).sum()         # validation log-likelihood
    val_ll /= N                                                # average per sample
    return Ks[int(np.argmax(val_ll))], val_ll


def run_experiment():
    print(f"distance between means of components 1 and 2 = {dist12:.3f}; "
          f"sum of their average covariance eigenvalues = {avg_eig[0] + avg_eig[1]:.3f}")
    t0 = time.time()
    sel = {}; curves = {}
    for N in Ns:
        out = Parallel(n_jobs=2)(delayed(one_run)(N, r) for r in range(R))
        sel[N] = np.array([o[0] for o in out]); curves[N] = np.array([o[1] for o in out])
        rate = np.array([np.mean(sel[N] == K) for K in Ks])
        print(f"N={N:5d}: selection rate by K = " +
              " ".join(f"{K}:{100*r:.0f}%" for K, r in zip(Ks, rate)) +
              f"   mode={Ks[int(np.argmax(rate))]}  mean={sel[N].mean():.2f}  [{time.time()-t0:.0f}s]",
              flush=True)

    # true-density reference: expected per-sample log-likelihood (entropy estimate)
    Xbig, _ = sample_gmm(200000, np.random.default_rng(1))
    ll_true = float(true_logpdf(Xbig).mean())
    print(f"E[log p_true(x)] ≈ {ll_true:.4f}")
```

```
    res = {"R": R, "Ns": Ns, "Ks": Ks, "dist12": float(dist12), "avg_eig": [float(a) for a in avg_eig],
           "ll_true": ll_true,
           "selected": {N: sel[N].tolist() for N in Ns},
           "rates": {N: [float(np.mean(sel[N] == K)) for K in Ks] for N in Ns}}
    with open("q2_results.json", "w") as f:
        json.dump(res, f, indent=1)
    np.savez("q2_curves.npz", **{f"N{N}": curves[N] for N in Ns})
    print(f"experiment done in {time.time()-t0:.0f}s")


def make_figures():
    """All figures are drawn from the saved results, so they can be redrawn without rerunning."""
    res = json.load(open("q2_results.json"))
    res["rates"] = {int(k): v for k, v in res["rates"].items()}
    ll_true = res["ll_true"]
    npz = np.load("q2_curves.npz"); curves = {N: npz[f"N{N}"] for N in Ns}
    # ---------------- figures ----------------
    # (a) true GMM and one dataset of each size
    gx, gy = np.meshgrid(np.linspace(-9, 11, 300), np.linspace(-4, 10, 220))
    Z = np.exp(true_logpdf(np.column_stack([gx.ravel(), gy.ravel()]))).reshape(gx.shape)
    fig, axes = plt.subplots(1, 3, figsize=(15, 4.4))
    for ax, N in zip(axes, Ns):
        X, comp = sample_gmm(N, np.random.default_rng([SEED, N, 0]))
        ax.contour(gx, gy, Z, levels=8, colors="gray", linewidths=0.8)
        for k, mk in zip(range(4), "o^sD"):
            ax.scatter(*X[comp == k].T, s=12 if N < 1000 else 5, marker=mk, label=f"component {k+1}")
        ax.plot(*m_true.T, "k+", ms=12, mew=2)
        ax.set_title(f"N = {N} (repetition 1)"); ax.set_aspect("equal")
        ax.set_xlabel("$x_1$"); ax.set_ylabel("$x_2$")
    axes[0].legend(fontsize=7, loc="upper right")
    fig.suptitle("Q2: true GMM density contours with sample datasets (components 1 and 2 overlap)")
    fig.tight_layout(); fig.savefig("figs/q2_data.png", dpi=140); plt.close(fig)

    # (b) selection-rate bar chart
    fig, ax = plt.subplots(figsize=(8, 4.5))
    wbar = 0.27
    for i, N in enumerate(Ns):
        r = np.array(res["rates"][N]) * 100
        ax.bar(np.array(Ks) + (i - 1) * wbar, r, wbar, label=f"N = {N}")
        for K, v in zip(Ks, r):
            if v >= 1:
                ax.text(K + (i - 1) * wbar, v + 1, f"{v:.0f}", ha="center", fontsize=7)
    ax.axvline(4, color="k", ls=":", lw=1); ax.text(4.05, 92, "true order", fontsize=8)
    ax.set_xticks(Ks); ax.set_xlabel("selected number of components K")
    ax.set_ylabel(f"selection rate over {R} repetitions (%)"); ax.set_ylim(0, 100)
    ax.set_title("Q2: GMM order selected by 10-fold cross-validation")
    ax.legend(); ax.grid(alpha=0.3, axis="y")
    fig.tight_layout(); fig.savefig("figs/q2_selection_rates.png", dpi=160); plt.close(fig)

    # (c) validation log-likelihood curves
    fig, axes = plt.subplots(1, 3, figsize=(15, 4.3))
    for ax, N in zip(axes, Ns):
        cv = curves[N].copy()
        cv_f = np.where(np.isfinite(cv), cv, np.nan)
        cv_f[:, np.all(np.isnan(cv_f), axis=0)] = np.nan
        med = np.nanmedian(cv_f, axis=0)
        lo, hi = np.nanpercentile(cv_f, [10, 90], axis=0)
        if N == 10:
            # held-out scores span three orders of magnitude here: plot the NEGATIVE
            # validation log-likelihood on a log axis (lower is better)
            ax.fill_between(Ks, -hi, -lo, alpha=0.25, label="10–90% across repetitions")
            ax.plot(Ks, -med, "o-", label="median")
            ax.axhline(-ll_true, color="r", ls="--", lw=1, label="true-pdf expected value")
            ax.set_yscale("log"); ax.set_ylabel("NEGATIVE avg. validation log-likelihood (log scale)")
            ax.text(9.6, -ll_true * 1.3, "K=10 infeasible\n(9 training points)", fontsize=7, ha="right")
        else:
            ax.fill_between(Ks, lo, hi, alpha=0.25, label="10–90% across repetitions")
            ax.plot(Ks, med, "o-", label="median")
```

```
            ax.axhline(ll_true, color="r", ls="--", lw=1, label="true-pdf expected log-lik.")
            ax.set_ylabel("average validation log-likelihood")
            ax.set_ylim(min(np.nanmin(lo[:6]), ll_true) - 0.3, max(np.nanmax(hi), ll_true) + 0.2)
        ax.axvline(4, color="k", ls=":", lw=1)
        ax.set_xticks(Ks); ax.set_xlabel("K"); ax.set_title(f"N = {N}")
        ax.grid(alpha=0.3)
    axes[0].legend(fontsize=8)
    fig.suptitle("Q2: 10-fold CV validation log-likelihood per sample vs. number of components")
    fig.tight_layout(); fig.savefig("figs/q2_cv_curves.png", dpi=140); plt.close(fig)


if __name__ == "__main__":
    # usage: python q2.py [R]        -> run experiment with R repetitions, then draw figures
    #        python q2.py --plots    -> redraw figures from saved results only
    if "--plots" not in sys.argv:
        run_experiment()
    make_figures()
```